VCET UniCore : AI-Powered Student Academic Portal

Paper based on the Project

RAUT CHINMAY NILESH
Department of Information Technology
Vidyavardhini's College of Engineering & Technology
K. T. Marg, Vasai Road (W), Palghar-401202
chinmay.2403674101@vcet.edu.in

SHINDE SHLOK MANOJ
Department of Information Technology
Vidyavardhini's College of Engineering & Technology
K. T. Marg, Vasai Road (W), Palghar-401202
Shlok.2403744102@vcet.edu.in

RAUT JAGRAVEE KALPAK
Department of Information Technology
Vidyavardhini's College of Engineering & Technology
K.T. Marg, Vasai Road (W), Palghar-401202
jagravee.2403684201@vcet.edu.in

RANE SANSKRUTI MANGESH
Department of Information Technology
Vidyavardhini's College of Engineering & Technology
K.T. Marg, Vasai Road (W), Palghar-401202
sanskruti.2403654210@vcet.edu.in

***Abstract*-- This paper presents the comprehensive development and implementation of VCET UniCore, an advanced, highly integrated AI-Powered Student Academic Portal designed to modernize educational management, analytics, and placement readiness tracking. The project delivers intuitive graphical interfaces for administrators, faculty, and students, enabling seamless bulk academic record ingestion, real-time attendance tracking, targeted announcements, and predictive academic modeling without the need for native mobile applications. Built using a robust backend framework in Spring Boot (Java) and a dynamic, rapid frontend powered by Vite, HTML5, and JavaScript, the application utilizes MySQL for complex relational database management and Weka for machine learning capabilities. The system guarantees secure data handling and authentication through Spring Security and JSON Web Tokens (JWT), alongside transactional email notifications via the Brevo API. The overarching objective is to eradicate the inefficiencies of traditional manual academic tracking, drastically reduce administrative entry errors through CSV bulk uploads,**

and provide a structured enterprise application demonstrating the intricate integration of RESTful APIs, Artificial Intelligence (RandomForest), and the Model-View-Controller (MVC) design principle.

I. INTRODUCTION

Manual academic management in engineering colleges tends to break down at scale. When attendance is tracked on paper, results are entered one by one, and there is no single place to see a student's full academic picture, things fall through the cracks sometimes quietly, until it's too late to course-correct. VCET UniCore was built to address exactly this. The idea was straightforward: take the core workflows of a college ecosystem onboarding, subject assignment, result computation, attendance, and performance prediction and tie them together into one coherent system.

The architecture keeps the three main user groups Admins, Faculty, and Students in clearly separated modules, so each role only ever sees what's relevant to them. Admins handle the institutional setup and data management. Faculty focus on attendance and assigned subjects. Students get a personal dashboard with their SGPA, CGPA, attendance, and AI-generated predictions about their future performance and placement eligibility. The technical backbone combines Weka for machine learning, Brevo for email dispatch, JWT and BCrypt for secure authentication, and Hibernate/JPA for database access all wired together through a standard MVC structure.

II. SYSTEM ARCHITECTURE

VCET UniCore is structured around the Model-View-Controller pattern, which in the Spring ecosystem naturally maps to clean separation between data, presentation, and business logic.

1. Users: The system serves three types of users are Admin, Faculty and Student. Each role has its own dedicated interface and can only access what is relevant to them, keeping the system clean and secure.

2. Frontend: It is built using Vite, HTML5, CSS3 and Vanilla JavaScript. It serves role-specific dashboards that call the backend REST APIs and render the returned data as charts, tables, and summary cards. No mobile app is needed.

3. Security: The Gatekeeper, Every incoming request carries a JWT token. Spring Security reads that token, checks your role, and blocks access if you try to reach a route you're not allowed on. Passwords are hashed using BCrypt and account claim or reset links are time-limited, dispatched via Brevo email.

4. Backend: The Engine The Spring Boot application (Java 21) is the core of the system. Controllers receive HTTP requests, validate the JWT, and pass work to service classes. Services handle all the real logic for computing SGPA and CGPA using MU NEP rules, parsing bulk CSV uploads, and building the JSON response sent back to the frontend.

5. AI Engine: The Predictor A background job runs every 6 hours, retraining a Weka RandomForest model on the latest result and attendance data. When a student opens their dashboard, the AIPredictionService loads the trained model and outputs a predicted final CGPA, next-semester SGPA, placement readiness score, and company tier eligibility

6. External Services: The Messenger When an admin bulk-imports students via CSV, the backend auto-generates a unique claim token per student and fires an HTML-formatted welcome email through the Brevo API. The same flow handles password resets using expiring secure links.

7. Database: The Memory MySQL stores all persistent data across 7 normalised tables: Students, Admins, Faculty, Subjects, Results, Attendance, and Announcements. Hibernate/JPA maps Java objects directly to SQL tables, so no raw queries exist in the codebase. Foreign keys maintain data integrity across related records.

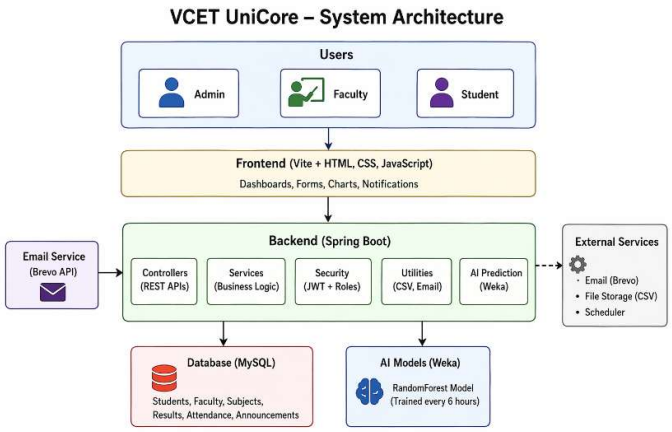

Fig.1 System Architecture

III. Modules of the System :

A. Admin Module

The admin interface functions as the institutional command center. From here, an admin can:

- Onboard students and faculty either individually or via bulk CSV upload.
- Create subjects, assign credits, and map faculty members to specific subjects for a given semester.
- Upload bulk result CSVs where the backend automatically computes grades, SGPA, and CGPA using the MU NEP grading scale.
- Send announcements to all students or to a specific subset.

B. Faculty Module

- Assigned Subjects View: Faculty can view the specific subjects and semesters they have been assigned to teach.
- Attendance Marking**:** An intuitive interface allowing faculty to select a date, session number, and subject to bulk-mark attendance (Present/Absent) for enrolled students, featuring upsert logic to prevent duplicate entries.
- Self-Service Security**:** Features for securely managing and changing their account passwords.

C. Student Module

Students get the most data-rich view. After claiming their account via an email token, they land on a dashboard showing:

- SGPA and CGPA progression through visual line charts.
- Per-subject grades, backlog count, and attendance percentages.
- AI-generated prediction predicts final CGPA, next-semester SGPA, placement readiness score, and company tier eligibility (Product vs. Service).
- A localized announcements inbox showing global and targeted messages.

D. Authentication & Security

The platform implements a highly secure, role-based authentication matrix:

- Stateless JWT Authentication: Handled via JSON Web Tokens embedded with user roles and IDs, completely eliminating server-side session state.
- Registration & Secure Login**:** Handled via secure credentials (BCrypt hashed passwords) stored in the database.
- Password Management & Claim Flow**:** A comprehensive, secure account claim and password reset flow utilizing Brevo-dispatched email tokens.
- Authorization Logic**:** Implemented using Spring Security. URL routes and API endpoints are strictly protected based on roles (ADMIN, FACULTY, STUDENT), preventing unauthorized access to sensitive academic data or administrative functions.

Table I. Tools and Technologies Used

Component	**Technology / Framework Used**	**Purpose in Project**
Frontend UI	HTML5, CSS3, Vanilla JavaScript	Building the core visual structure, styling, and client-side interactivity.
Frontend Build	Vite	Rapid development server and asset bundling.
Backend Core	Java 21	The primary programming language providing robust
Backend Framework	Spring Boot 4.0.3	Rapid application development framework utilizing Web, Data JPA, and Security starters.
Database	MySQL	Relational database management system for persistent, structured data storage.
AI/ML Engine	Weka	Implementing and training RandomForest models for academic predictions.
Build Tool	Maven (pom.xml)	Managing project dependencies, builds, and lifecycle management.

IV. Libraries Used

- **Spring Boot Web:** For building RESTful APIs and Spring MVC controllers.
- **Spring Boot Data JPA & Hibernate:** For Object-Relational Mapping, translating Java classes into SQL tables and managing database transactions.
- **Spring Boot Security:** For implementing robust authentication and authorization protocols.
- **Weka 3.8:** Core machine learning library utilized for data pre-processing, training the RandomForest algorithms, and serializing models.
- **Brevo (SendinBlue) API:** For dispatching transactional emails (e.g., account claim links, password reset tokens) via HTTP REST calls.
- **JJWT (JSON Web Token):** Utilized for secure, stateless API authentication, token generation, and signature verification.
- **Jackson:** The core library for serializing and deserializing Java objects into JSON data for API responses

V. DATABASE STRUCTURE

The database follows a normalized relational schema. The main tables and their roles are:

1. **Students (students)**
   Stores student profiles and credentials. Fields: id, name, email (unique), department, semester, password, isActive, claimToken.

2. **Admins (admins)**
   Stores super-user credentials. Fields: id, email (unique), password.

3. **Faculty (faculty)**
   Stores teaching staff profiles. Fields: id, name, email (unique), password, department.

4. **Subjects (subjects)**
   Maps curriculum to faculty. Fields: id, subjectName, credits, semester, faculty_id (FK to Faculty).

5. **Results (results)**
   The core academic ledger. Fields: id, student_id (FK), courseCode, courseName, internalMarks,externalMarks, totalMarks, credits, grade, gradePoint, semester.

6. **Attendance (attendance)**
   Tracks granular session-level attendance. Fields: id, student_id (FK), subject_id (FK), faculty_id (FK), date, status (PRESENT/ABSENT), sessionNumber.

7. **Announcements (announcements)**
   Stores broadcasts. Fields: id, title, message, targetType (ALL/SELECTED), targetStudentIds, createdAt.

VI. Process and Working

The typical lifecycle of the system from initial setup to day-to-day use unfolds as follows:

1. System Onboarding: The admin logs in and mass-imports student and faculty data through formatted CSV files. The system auto-generates claimTokens and fires off welcome emails via Brevo.

2. Account Claiming: Students and faculty receive an email with a secure link, visit the portal, and use their token to set a permanent password.

3. Curriculum Setup: Admins define subjects, assign credits, and link specific faculty members for the active semester.

4. Academic Tracking: Throughout the semester, faculty mark daily attendance through their portal.

5. Result Compilation: After exams, admins upload marks via CSV. The backend calculates totals, applies MU NEP grading, and computes SGPA and CGPA automatically.

6. AI Pipeline Execution: A background service pulls the latest result and attendance data, engineers features, and retrains the Weka RandomForest models every 6 hours, serializing them to disk.

7. Data Visualization & Prediction: Students log into their dashboards. REST APIs feed the Vite frontend with analytics. The AIPredictionService evaluates their data against the trained models and surfaces predicted CGPA, next-semester SGPA, and placement readiness.

VII. Features

- **Frictionless CSV Ingestion:** Rapid onboarding and result entry through standardized bulk CSV processing, drastically reducing manual data entry.

- **Automated MU NEP Grading:** Instant, mathematically accurate calculation of grades, SGPA, and CGPA based on the Mumbai University National Education Policy guidelines.

- **Role-Based Dashboards:** Distinct, secure, and highly contextual interfaces tailored specifically for the operational needs of Admins, Faculty, and Students.

- **Targeted Communication System:** A built-in announcement platform supporting both global broadcasts and localized messages to specific student groups.

Special Features

- **AI-Powered Predictive Analytics:** Utilizes Weka Machine Learning (RandomForest) to analyze past performance and attendance to predict a student's final CGPA and next-semester SGPA.
- **Placement Readiness Scoring:** An algorithmic evaluation that combines AI predictions, backlog counts, and academic consistency to generate a standardized placement probability and eligibility for Product vs. Service companies.

- **Transactional Email Integration:** Seamless backend integration with the Brevo API to automatically dispatch HTML-formatted welcome emails, claim codes, and password reset links.

- **Stateless Security Architecture:** High-performance JWT-based authentication ensuring secure, scalable, and session-less API communications.

VIII. Detailed User Interface Description

The VCET UniCore system comprises multiple meticulously designed interfaces tailored for specific user roles. Below is a breakdown of the primary views.

➢ Landing Page

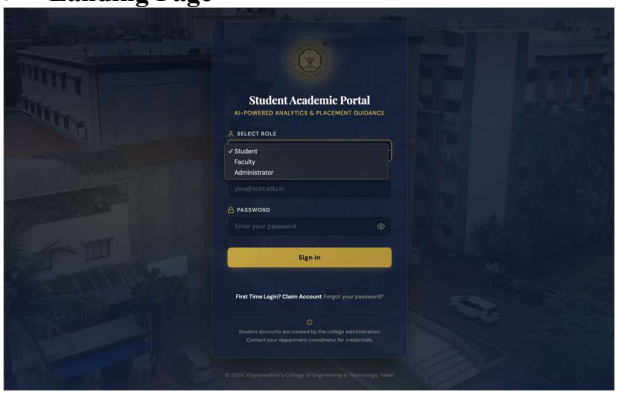

Fig.2 Sign In Dashbord

- **Purpose:** The secure entry point for all users.
- **Components:** A modern interface featuring role selection (Student, Faculty, Admin), standard credential inputs, and access to "Claim Account" and "Forgot Password" modals.

➢ Admin Dashboard

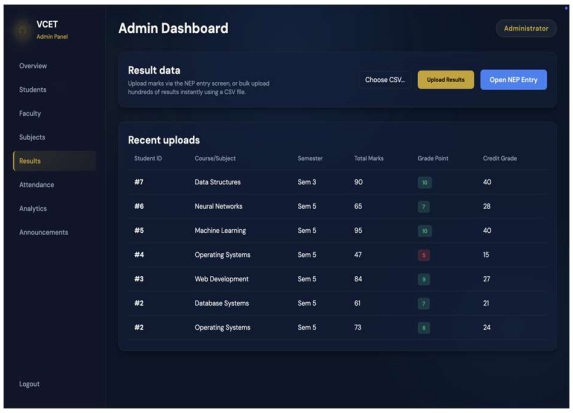

Fig.3 admin-dashboard.html

- **Purpose:** The central command hub for institutional management.
- **Components:** Navigation tabs for Student Management, Faculty Management, Subjects, Results, and Announcements. Includes interactive data tables, CSV upload dropzones, and forms for assigning faculty to subjects.

➢ Faculty Dashboard

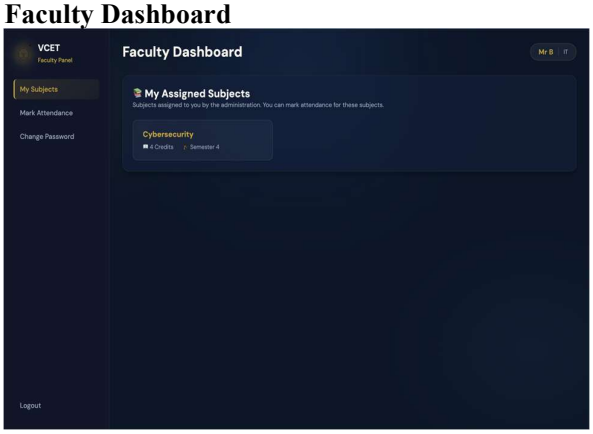

Fig.4 Faculty-dashboard.html & attendance marking.html

- **Purpose:** Streamlined interface for academic staff.
- **Components:** A view of assigned curriculum. The core feature is the Attendance Marking grid, where faculty select a date and subject, presenting a list of enrolled students with quick-toggle Present/Absent buttons.

➢ Student Dashboard

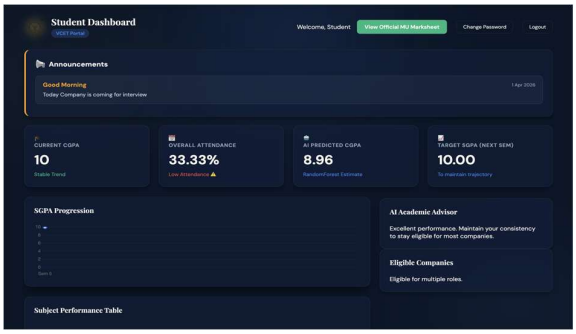

Fig.5 Student-dashboard.html & result-page.html

- **Purpose:** The analytical view for the student's academic profile.
- **Components:** High-level summary cards (Current SGPA, Total Credits). Interactive line charts mapping SGPA progression. A dedicated "AI Advisor" section displaying predicted metrics, placement readiness scores, and personalized insights regarding strong and weak subjects. A historical result grid detailing individual subject grades.

IX. Testing and Results

- **Authentication & Role Testing:** Attempts to access /api/admin/* routes using a student JWT consistently returned HTTP 403 Forbidden. Faculty and student records remained fully isolated from each other.

- **AI Model Viability Testing:** The Weka pipeline was tested with synthetic historical data. The RandomForest model correctly identified performance patterns and produced statistically reasonable CGPA predictions, updating as expected after each 6-hour retrain cycle.

- **Bulk Upload Testing:** Multi-hundred-row CSV files were processed without connection timeouts. Malformed rows were skipped, MU NEP grade calculations were correct, and all valid data was persisted to MySQL.

- **Email Delivery Verification:** Account claim workflows were triggered and tested end-to-end. Brevo-dispatched emails arrived with correctly generated, time-sensitive tokens.

X. Future Scope

- **Parental Portal Integration:** A read-only access tier for parents to track their ward's academic progress and attendance without interfering with the main system.

- **Deep Learning Enhancements:** Replacing the RandomForest models with LSTM-based neural networks to better handle the sequential, time-series nature of academic data.

- **Library & Fee Management:** Expanding the platform into a full college ERP by adding modules for library tracking and tuition fee payment.

XI. Conclusion

The VCET UniCore system effectively demonstrates how modern web technologies, AI integration, and enterprise Java frameworks can be leveraged to fundamentally streamline university academic management. The technical combination of Spring Boot's robust backend capabilities, Vite's dynamic rendering, Weka's machine learning, and MySQL provides a scalable, highly intelligent architecture. This project not only automates historically manual, error-prone tasks like result calculation and attendance tracking but also equips students with proactive, AI-driven insights regarding their placement readiness. It successfully fulfills its overarching goal of delivering a comprehensive, data-driven, and highly modern digital academic solution.

References

[1] V. Kassarnig, D. Koutra, I. Elmakies, et al., "Class Attendance, Peer Similarity, and Academic Performance," 2017. [Online]. Available: https://arxiv.org/abs/1702.01262.

[2] B. Albreiki, N. Zaki, and H. Alashwal, "A Systematic Literature Review of Student Performance Prediction Using Machine Learning Techniques," Education Sciences, vol. 11, no. 9, 2021. [Online]. Available: https://www.mdpi.com/2227-7102/11/9/552.

[3] B. K. Baradwaj and S. Pal, "Mining Educational Data to Analyze Students' Performance," 2011. [Online]. Available: https://arxiv.org/abs/1201.3417.

[4] Y. Li, "Student Academic Performance Prediction Based on Machine Learning Algorithms," 2024. [Online]. Available: https://www.ewadirect.com/proceedings/chr/article/view/11769

[5] R. Elhassan and A. Alqurashi, "Educational Data Mining for Student Performance Prediction," 2022. [Online]. Available: https://slejournal.springeropen.com/articles/10.1186/s40561-022-00192-z